

Documentation de configuration de l'environnement de travail

Date de création : 05/10/2026 - E. BAYRAM

Date de modification : 05/10/2026 - E. BAYRAM

Sommaire :

[TOC]

1. Logiciels et composants utilisés

Pour assurer le développement et l'exécution de l'application Web QCM, l'environnement de travail repose sur une pile LAMP sous WSL (Windows Subsystem for Linux) :

- **Système d'exploitation** : Ubuntu 22.04 LTS (via WSL 2)
 - **Serveur Web** : Apache / 2.4.52
 - **Langage Backend** : PHP / 8.1.x (avec extensions pdo, pdo_mysql, mbstring, xml)
 - **Système de Gestion de Base de Données (SGBD)** : MariaDB / 10.6.x
 - **Gestionnaire de dépendances** : Composer / 2.x
 - **Génération de documentation technique** : Doxygen / PHPDoc
 - **Éditeur de code** : Visual Studio Code (avec extension WSL Remote)
 - **Terminal SSH / CLI** : VS Code Integrated Terminal / Windows Terminal
 - **Client SFTP / Transfert** : VS Code Remote / FileZilla 3.x
 - **Gestionnaire de version** : Git / 2.34.x
-

2. Procédure d'installation de l'environnement (Local / WSL)

Voici les commandes d'installation et de configuration de la pile sur une machine Ubuntu / WSL vierge :

2.1. Mise à jour du système

```
sudo apt update && sudo apt upgrade -y
```

2.2. Installation d'Apache, MariaDB, PHP et Doxygen

```
sudo apt install -y apache2 mariadb-server php libapache2-mod-php php-mysql php-cli php-curl php-gd  
php-mbstring php-xml php-zip composer git doxygen
```

2.3. Activer le module d'écriture d'URL Apache (mod_rewrite)

```
sudo a2enmod rewrite sudo systemctl restart apache2
```

3. Workflow GitFlow et branches du projet

Le suivi de version respecte la logique du framework GitFlow :

- **main** : Branche réservée aux versions stables et validées (prêtes pour la production, marquées par des tags de version).
 - **develop** : Branche principale d'intégration et de travail centralisée.
 - **feat/<nom-fonctionnalité>** : Branches éphémères pour développer des fonctionnalités spécifiques avant fusion vers develop.
-

4. Architecture des répertoires et mesures de sécurité

L'arborescence respecte la structure MVC imposée par le sujet :

Cassin-SIO26-AP3-QCM-BAYRAM/ | — app/ # Code applicatif non accessible directement depuis le Web | | — Controllers/ # Contrôleurs de l'application | | — Models/ # Modèles de données | | — Views/ # Vues HTML/PHP | | — Middleware/ # Filtres de sécurité (CSRF, Authentification, Droits) | | — Security/ # Authentification et autorisations | | — Core/ # Cœur du framework (Router, Database) | — public/ # Seul dossier accessible publiquement par le serveur Web | | — index.php # Point d'entrée unique (Front Controller) | | — assets/ # Fichiers statiques (CSS, JS, images) | | — .htaccess # Redirection de toutes les requêtes vers index.php | — config/ # Fichiers de configuration (accès BDD) | — docs/ # Documentations du projet (Markdown) | — storage/ # Logs et données système | — uploads/ # Fichiers téléversés hors du répertoire web public | — tests/ # Tests unitaires

Sécurité du répertoire :

- **Point d'entrée unique** : Seul le répertoire public/ est exposé au serveur Apache. Les dossiers sensibles (app/, config/, docs/) ne sont pas accessibles via l'URL pour empêcher toute fuite de code ou d'identifiants.
- **Masquage de la configuration** : Les identifiants BDD sont stockés dans le dossier config/ situé en dehors de la racine web public/.

5. Procédure de déploiement vers l'environnement de tests

Pour déployer une nouvelle version du code sur l'environnement de test :

1. Se placer dans le répertoire du projet : `cd /var/www/html/Cassin-SIO26-AP3-QCM-BAYRAM`
2. Récupérer les dernières modifications de la branche develop (ou main pour la production) : `git checkout develop git pull origin develop`
3. Mettre à jour les dépendances si nécessaire : `composer install --no-dev --optimize-autoloader`
4. Ajuster les permissions sur les dossiers d'écriture : `chmod -R 775 storage uploads`

6. Conventions de nommage des commits (Conventional Commits)

Les messages de validation Git doivent obligatoirement respecter la norme Conventional Commits : ():

- **feat** : Nouvelle fonctionnalité (ex: feat(login): ajout du formulaire de connexion)
- **fix** : Correction de bug (ex: fix(router): correction de la redirection 404)
- **docs** : Documentation uniquement (ex: docs(technique): mise à jour du guide d'installation)
- **style** : Changement d'affichage, CSS/HTML sans modifier la logique (ex: style(css): modification de la couleur des boutons)
- **refactor** : Modification du code sans ajout de fonction ni correction (ex: refactor(database): optimisation de la connexion PDO)
- **chore** : Tâches de configuration ou maintenance (ex: chore(config): ajout du fichier .gitignore)